

DP: Subșir Comun Maximal și Problema Rucsacului

SCM (LCS) · Knapsack · Subset Sum

Andrei Mihai Ciobanu

Algolymp Summer School

Cuprins

Subșir comun maximal (SCM)

Problema Rucsacului

Recapitulare

Subșir comun maximal (SCM)

Definiție

Subșir

Un **subșir** al lui $A[1..n]$ se obține alegând indici $i_1 < i_2 < \cdots < i_k$ și păstrând elementele $A[i_1], \ldots, A[i_k]$. Ordinea relativă e păstrată; elementele **nu** trebuie să fie consecutive.

Problema SCM

Se dau șirurile $A[1..n]$ și $B[1..m]$. Găsiți lungimea celui mai lung șir care este subșir atât al lui A, cât și al lui B.

Exemplu: $A =$ `ABCBD`, $B =$ `BDCAB`

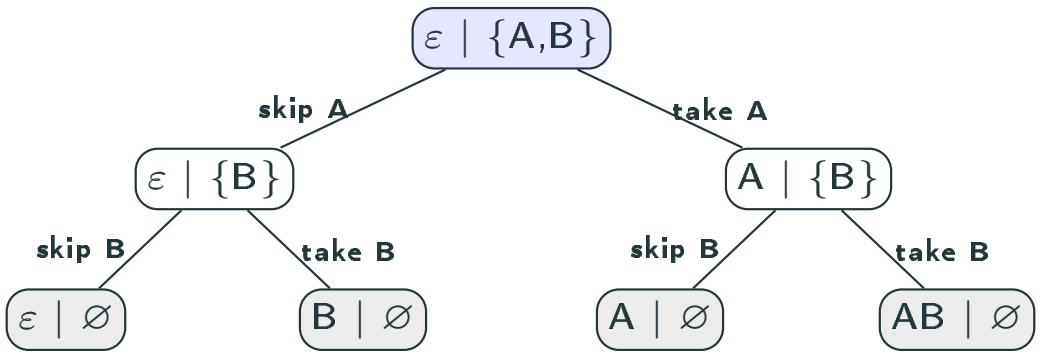
- Subșiruri comune: `B`, `BD`, `BC`, `BCB` ...
- Lungime maximă: 3 (`BCB`)

Aplicații: diff între fișiere, comparare secvențe ADN, detecție plagiat.

Abordare bruteforce

Idee: la fiecare poziție din A decidem *includem* sau *sărim* elementul — generăm toate subșirurile lui A prin backtracking, iar fiecare subșir obținut e verificat în B.

Fiecare nod are forma **ales** | **{rămase}**: subșirul construit până acum | caracterele din A încă neprocesate. (ε = șirul vid)



A are 2^n subșiruri, fiecare verificat în $O(n+m)$ $\Rightarrow$ complexitate totală $O(2^n \cdot (n+m))$, exponențial.

Starea DP și recurența

Stare

$dp[i][j]$ = lungimea SCM dintre $A[1..i]$ și $B[1..j]$

Recurență

$$dp[i][j] = \begin{cases} dp[i-1][j-1] + 1 & \text{dacă } A[i] = B[j] \\ \max(dp[i-1][j],\ dp[i][j-1]) & \text{altfel} \end{cases}$$

Bază: $dp[i][0] = dp[0][j] = 0$

Intuiție:

- $A[i] = B[j]$: ambele caractere fac parte din SCM $\Rightarrow$ extindem cu $+1$ față de $dp[i-1][j-1]$
- $A[i] \neq B[j]$: renunțăm la $A[i]$ sau la $B[j]$, luăm maximul

Complexitate: $O(n \cdot m)$ timp și spațiu.

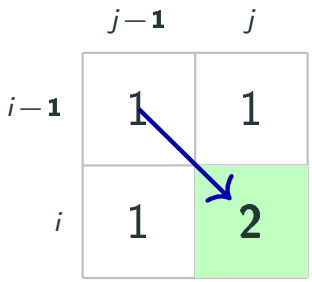
Ilustrare — cele două cazuri

Exemplu: $A =$ `ABCD`, $B =$ `ABDC` (celulele $dp[2][2]$ și $dp[2][3]$).

Cazul 1: $A[i] = B[j]$

i=2, j=2: B = B

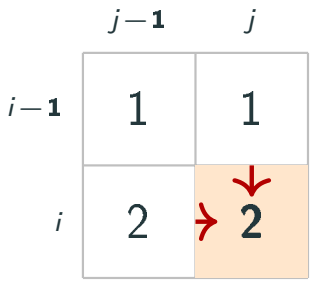
$dp[2][2] = dp[1][1] + 1 = 2$



Cazul 2: $A[i] \neq B[j]$

i=2, j=3: B ≠ D

$dp[2][3] = \max(dp[1][3], dp[2][2]) = 2$



Exemplu pas cu pas

$A =$ ABCD, $B =$ ABDC

dp	$\varnothing$	A	B	D	C
$\varnothing$	0	0	0	0	0
A	0	**1**	1	1	1
B	0	1	**2**	2	2
C	0	1	2	2	**3**
D	0	1	2	**3**	3

Celule bold = potrivire $A[i] = B[j]$, extindem diagonal $(+1)$.
Celulele fără potrivire = maximul din stânga sau de sus. Răspuns: $dp[4][4] = 3$

— SCM poate fi ABC sau ABD.

Implementare

```
1  const int nmax = 1000;
2  int dp[nmax + 1][nmax + 1];
3  char A[nmax + 1], B[nmax + 1];
4
5  // n, m, A[], B[] citite anterior (indexate de la 1)
6  for (int i = 1; i <= n; i++) {
7      for (int j = 1; j <= m; j++) {
8          if (A[i] == B[j]) {
9              dp[i][j] = dp[i - 1][j - 1] + 1;
10         } else {
11             dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
12         }
13     }
14 }
15 // Raspuns: dp[n][m]
```

Obs: $dp[i][0]$ și $dp[0][j]$ sunt implicit 0 (variabile globale).

Reconstrucția soluției

Idee: parcurgem dp înapoi de la (n, m) spre $(0, 0)$:

- $A[i] = B[j]$: caracterul face parte din SCM $\Rightarrow$ mergem diagonal
- Altfel: mergem spre celula mai mare (sus sau stânga)

```
string scm = "";
int i = n, j = m;
while (i > 0 && j > 0) {
    if (A[i] == B[j]) {
        scm += A[i];
        i--;
        j--;
    } else if (dp[i - 1][j] >= dp[i][j - 1]) {
        i--;
    } else {
        j--;
    }
}
reverse(scm.begin(), scm.end());
```

Reconstrucția — traseu în tabel

$A =$ ABCD, $B =$ ABDC: pornim din $dp[4][4]$, mergem înapoi.

	∅	A	B	D	C
∅	0	0	0	0	0
A	0	1	1	1	1
B	0	1	2	2	2
C	0	1	2	2	3
D	0	1	2	3	3

1. $(4,4)$: D≠C
   ↑ sus la $(3,4)$
2. $(3,4)$: C=C ✓
   ↖ diagonal la $(2,3)$
3. $(2,3)$: B≠D
   ← stânga la $(2,2)$
4. $(2,2)$: B=B ✓
   ↖ diagonal la $(1,1)$
5. $(1,1)$: A=A ✓
   ↖ diagonal la $(0,0)$

scm = CBA $\xrightarrow{\text{reverse}}$ **ABC**

potrivire fără potrivire ↗ diagonal →/↑ stânga/sus

Problema Rucsacului

Rucsac fractionar

Variantă

Putem lua **fracțiuni** din obiecte: fracțiunea $f_i \in [0, 1]$ din obiectul i adaugă greutate $f_i \cdot g_i$ și valoare $f_i \cdot v_i$.

Greedy: sortăm descrescător după v_i/g_i și luăm cât putem din fiecare.

Obiect	g_i	v_i	v_i/g_i	Luat
A	10	60	6.0	tot (10 kg)
B	20	100	5.0	tot (20 kg)
C	30	120	4.0	$\frac{2}{3}$ (20 kg din 30)

$W = 50$: valoare $= 60 + 100 + 80 = 240$. **Complexitate:** $O(n \log n)$. **De ce greedy nu merge pentru 0/1:** nu putem lua fracțiuni — fiecare obiect e întreg sau absent. Un obiect greu cu raport bun poate „bloca" combinații mai valoroase.

Rucsac 0/1 — backtracking

Problemă

Se dau n obiecte cu greutăți g_i și valori v_i, capacitate W. Fiecare obiect poate fi luat **cel mult o dată**. Maximizați valoarea.

Backtracking: la fiecare obiect decidem *luăm* sau *nu luăm*.

- 2^n submulțimi posibile $\Rightarrow$ complexitate $O(2^n)$, inacceptabil
- Aceeași stare (i, w) se recalculează exponențial de multe ori

Soluție DP: definim $dp[i][w]$ = valoarea maximă folosind primele i obiecte cu capacitate w. Calculăm fiecare stare **o singură dată** $\Rightarrow O(n \cdot W)$.

Rucsac 0/1 — starea DP și recurența

Stare

$dp[i][w]$ = valoarea maximă folosind primele i obiecte cu capacitate w

Recurență

$$dp[i][w] = \begin{cases} dp[i-1][w] & \text{dacă } g_i > w \text{ (nu încape)} \\ \max\big(dp[i-1][w],\ dp[i-1][w-g_i] + v_i\big) & \text{altfel} \end{cases}$$

Bază: $dp[0][w] = 0$ pentru orice w. **Răspuns:** $dp[n][W]$.

Intuiție:

- **Nu luăm obiectul** i: $dp[i-1][w]$ — aceeași capacitate, fără obiectul i
- **Luăm obiectul** i (dacă $g_i \le w$): $dp[i-1][w-g_i] + v_i$

Complexitate: $O(n \cdot W)$ timp, $O(n \cdot W)$ spațiu.

Rucsac 0/1 — implementare 2D

```
const int nmax = 5000;
const int wmax = 10000;
int g[nmax + 1], v[nmax + 1];
int dp[nmax + 1][wmax + 1];

// n, W, g[], v[] citite (indexate de la 1)
for (int i = 1; i <= n; i++) {
    for (int w = 0; w <= W; w++) {
        dp[i][w] = dp[i - 1][w];
        if (g[i] <= w) {
            dp[i][w] = max(dp[i][w], dp[i - 1][w - g[i]] + v[i]);
        }
    }
}
// Raspuns: dp[n][W]
```

Obs: pentru $n = 5000$, $W = 10000$ rezultă $5 \cdot 10^7$ celule — adesea prea mult ca memorie. Folosim optimizarea 1D.

Rucsac 0/1 — optimizare memorie $O(W)$

La calculul rândului i avem nevoie **doar de rândul** $i-1 \Rightarrow$ reducem la un singur vector, parcurgând **de la dreapta la stânga**:

```
const int wmax = 10000;
int dp[wmax + 1];

for (int i = 1; i <= n; i++) {
    for (int w = W; w >= g[i]; w--) {
        dp[w] = max(dp[w], dp[w - g[i]] + v[i]);
    }
}
// Raspuns: dp[W]
```

De ce dreapta la stânga?

La pasul i, $dp[w - g_i]$ trebuie să fie valoarea **înainte** de procesarea obiectului i (rândul $i-1$). Parcurgând descrescător, $dp[w - g_i]$ nu e încă suprascris cu valoarea de la pasul i, deci fiecare obiect e luat **cel mult o dată**.

Rucsac unbounded — definiție și recurență

Variantă

Fiecare obiect poate fi luat de **oricâte ori** (copii nelimitate).

Recurență 2D:

$$dp[i][w] = \begin{cases} dp[i-1][w] & \text{dacă } g_i > w \\ \max\big(dp[i-1][w],\ dp[i][w-g_i] + v_i\big) & \text{altfel} \end{cases}$$

Diferența față de 0/1: folosim $dp[i][w-g_i]$ (același rând i, nu $i-1$) — aceasta permite refolosirea obiectului i în aceeași selecție. **Consecință pentru 1D:** parcurgem w **de la stânga la dreapta** (crescător), astfel $dp[w-g_i]$ e deja actualizat pentru obiectul i când ajungem la w.

$$dp[w] = \max_{i:\, g_i \le w} \big(dp[w-g_i] + v_i\big)$$

Complexitate: $O(n \cdot W)$ timp, $O(W)$ spațiu (1D direct).

Rucsac unbounded — implementare $O(W)$

```cpp
const int wmax = 10000;
int dp[wmax + 1]; // dp[0] = 0

for (int i = 1; i <= n; i++) {
    for (int w = g[i]; w <= W; w++) { // stanga la dreapta!
        dp[w] = max(dp[w], dp[w - g[i]] + v[i]);
    }
}
// Raspuns: dp[W]
```

Comparație directă cu 0/1:

Variantă	Direcție buclă w	De ce?
0/1	$W \to g_i$ (descrescător)	$dp[w - g_i]$ să nu fie încă actualizat
Unbounded	$g_i \to W$ (crescător)	$dp[w - g_i]$ să fie deja actualizat

Subset Sum — definiție

Problemă

Date n numere întregi $a[1..n]$, există o submulțime cu suma **exact** S?

Legătura cu rucsacul 0/1: Subset Sum este un caz particular unde $g_i = v_i = a[i]$ și vrem să atingem valoarea exactă S. **Stare:** $dp[w] =$ `true` dacă există o submulțime cu suma w.

$$dp[w] = dp[w] \text{ OR } dp[w - a[i]]$$

Bază: $dp[0] =$ true; $dp[w] =$ false pentru $w > 0$. **Răspuns:** $dp[S]$. **Variante frecvente:**

- **Număr de submulțimi** cu suma S: `bool` $\rightarrow$ `long long`, OR $\rightarrow +=$
- **Valoare maximă** $\leq S$: cel mai mare $w \leq S$ cu $dp[w] =$ true
- **Toate sumele posibile**: $dp[w]$ rămâne `true` pentru toate sumele atinse

Subset Sum — implementare

```
const int smax = 100000;
bool dp[smax + 1];

dp[0] = true;
for (int i = 1; i <= n; i++) {
    for (int w = S; w >= a[i]; w--) { // invers (0/1, fiecare o singura data)
        dp[w] |= dp[w - a[i]];
    }
}
// Raspuns: dp[S]
```

Obs: dacă fiecare element poate fi folosit de mai multe ori (Subset Sum unbounded):

```
for (int i = 1; i <= n; i++) {
    for (int w = a[i]; w <= S; w++) { // crescator
        dp[w] |= dp[w - a[i]];
    }
}
```

Complexitate: $O(n \cdot S)$ timp, $O(S)$ spațiu.

Optimizări

Bitset — Subset Sum în $O(nS/64)$

Reprezentăm dp ca `bitset`: bitul $w = 1$ dacă suma w e posibilă. Tranziția $dp[w] \mid= dp[w - a[i]]$ devine un **shift** și **OR** pe tot bitset-ul simultan:

```
bitset<100001> dp;
dp[0] = 1;
for (int i = 1; i <= n; i++) {
    dp |= (dp << a[i]); // O(S/64) per element
}
// Raspuns: dp[S]
```

Grupare pe puteri de 2 — obiecte egale în cantitate mare

Dacă avem k copii ale unui obiect cu greutate g, în loc de k tranziții identice le grupăm în $\lceil \log_2 k \rceil + 1$ **pachete**: $1, 2, 4, \ldots,$ rest. Fiecare pachet e tratat ca un obiect 0/1 separat. Numărul de tranziții scade de la $O(k)$ la $O(\log k)$ per grup.

Recapitulare

Probleme de antrenament

1. **rucsac** (infoarena) — 0/1 Knapsack clasic
2. **ghiozdan** (pbinfo #819) — 0/1 Knapsack
3. **Book Shop** (CSES #1158) — 0/1 Knapsack
4. **Minimizing Coins** (CSES #1634) — Unbounded — nr. minim de monede pentru suma X
5. **Coin Combinations II** (CSES #1636) — Unbounded — câte moduri neordonate
6. **Money Sums** (CSES #1745) — Subset Sum — toate sumele posibile
7. **Woodcutters** (CF #545C) — SCM cu condiții suplimentare

Resurse

1. **pbinfo** — Cel mai lung subșir comun (în română)
2. **pbinfo** — Problema Rucsacului (în română)
3. **GeeksforGeeks** — 0/1 Knapsack — recursiv, memo, tabelar, $O(W)$
4. **USACO Guide** — Knapsack DP — 0/1, Subset Sum, probleme Gold
5. **CP-Algorithms** — Knapsack Problems — toate variantele
6. **Codeforces blog #98663** — Knapsack, Subset Sum — optimizări avansate cu bitset